

RIST 보고서 DB 전송 큐 연동 명세

대상: Spring Boot/LIMS 인터페이스 개발자

버전: 2.0

기준일: 2026-07-20

연동 방식: 공유 저장소 + MariaDB 전송 큐

1. 문서 목적

이 문서는 Edge가 생성한 최종 보고서를 Spring Boot가 LIMS로 전달하기 위한 전체 동작 사이클과 DB 계약을 정의한다.

Spring Boot는 Edge로부터 ZIP을 HTTP로 수신하거나 별도 디렉터리로 복사하지 않는다. Edge가 공유 저장소에 한 번 저장한 최종 ZIP을 MariaDB에 등록하면, Spring Boot 스케줄러가 DB 큐를 선점하고 같은 공유 저장소에서 파일을 직접 읽어 LIMS로 전송한다.

실험 PC/C#

- > Edge 작업 등록 및 raw 업로드
- > Edge 보고서 생성 요청
- > Edge가 공유 저장소에 report-package.zip 확정
- > Edge가 report_runs + report_transfers(PENDING) 등록
- > Spring Boot 스케줄러가 전송 작업 선점
- > 공유 저장소 ZIP 크기/SHA-256/형식 검증
- > LIMS 전송
- > report_transfers + report_transfer_attempts 결과 기록

이 문서에서 정의하지 않는 LIMS 수신 API의 URL, 인증, multipart 필드명과 응답 형식은 LIMS 인터페이스 계약에서 별도로 확정한다.

2. 핵심 설계 원칙

1. ZIP은 공유 저장소에 한 번만 저장한다.
2. DB에는 ZIP 본문, 절대 경로, `file://` URI를 저장하지 않는다.
3. DB에는 `storage_key + package_relative_path` 만 저장한다.
4. Edge 작업 완료와 LIMS 전송 완료는 별도 상태로 관리한다.
5. Spring Boot는 스케줄러를 여러 인스턴스로 실행해도 같은 작업을 동시에 처리하지 않아야 한다.
6. 파일 I/O와 LIMS 통신 중에는 DB 행 잠금을 유지하지 않는다.
7. 모든 전송 시도는 감사 가능한 이력으로 남긴다.
8. 전송 중인 ZIP은 삭제하거나 같은 경로에 덮어쓰지 않는다.

3. 범위와 비범위

3.1 포함 범위

- Edge 보고서 패키지 등록 계약
- Spring Boot 큐 조회, 선점, lease, 재시도와 복구
- 공유 저장소 경로 해석과 파일 무결성 검증
- 전송 성공, 실패와 시도 이력 저장
- 운영 조회, 수동 재시도와 취소 기준
- 배포 설정과 인수 테스트 기준

3.2 포함하지 않는 범위

- 실험 PC가 Edge에 raw 파일을 올리는 기존 API
- Edge 내부 분석용 `report.json`, LLM 로그와 중간 파일
- Spring Boot가 ZIP을 수신하는 HTTP API
- LIMS 제품별 수신 API 세부 계약
- LIMS 내부 보고서 승인 및 업무 완료 처리

4. 구성요소별 책임

구성요소	책임
실험 PC/C#	Edge 작업 생성, raw 업로드, 업로드 완료, 보고서 생성 요청, Edge 상태 조회
Edge API/worker	보고서 생성, 공유 ZIP 확정, 크기/SHA-256/ZIP 검증, DB 큐 등록
MariaDB	보고서 위치와 무결성 메타데이터, 전송 상태, 시도 이력 보관
공유 저장소	Edge와 Spring Boot가 함께 접근하는 최종 ZIP 원본 보관
Spring Boot scheduler	큐 선점, 파일 재검증, LIMS 전송, 결과와 재시도 상태 기록
LIMS	최종 ZIP 수신, 업무키 검증, 처리 결과 반환

Spring Boot는 `jobs.status` 를 변경하지 않는다. `jobs` 는 Edge 작업 상태이고, LIMS 전송 상태는 `report_transfers.status` 로 판단한다.

5. 전체 동작 사이클

5.1 Edge 작업 단계

1. 실험 PC가 `POST /api/v1/jobs` 로 Edge 작업을 등록한다.
2. 실험 PC가 `POST /api/v1/jobs/{jobId}/files` 로 raw 파일을 업로드한다.
3. 실험 PC가 `POST /api/v1/jobs/{jobId}/uploads/complete` 를 호출한다.

4. 실험 PC가 `POST /api/v1/jobs/{jobId}/report` 를 호출한다.
5. Edge worker가 분석과 렌더링을 수행한다.
6. Edge가 최종 `report-package.zip` 을 공유 저장소 안에 확정한다.
7. Edge가 ZIP 형식, 파일 크기와 SHA-256을 검증한다.
8. Edge가 하나의 DB 트랜잭션으로 `report_runs` 와 `report_transfers(PENDING)` 를 등록한다.
9. 큐 등록까지 성공하면 Edge가 `jobs.status=COMPLETED` 로 변경한다.

`jobs.COMPLETED` 는 보고서 생성, 공유 저장소 게시와 DB 큐 등록 완료를 의미한다. LIMS 전송 완료를 의미하지 않는다.

5.2 Spring Boot 전송 단계

1. 스케줄러가 처리 가능한 전송 큐를 주기적으로 조회한다.
2. 짧은 트랜잭션에서 `FOR UPDATE SKIP LOCKED` 로 작업을 선점한다.
3. 상태를 `PROCESSING` 으로 바꾸고 lease와 시도 이력을 생성한다.
4. DB 트랜잭션을 종료한다.
5. `storage_key` 를 Spring 서버의 로컬 공유 저장소 루트에 매핑한다.
6. 상대 경로를 정규화하고 저장소 루트 탈출 여부를 검증한다.
7. 실제 파일 크기, SHA-256과 ZIP 형식을 검증한다.
8. 원본 위치에서 ZIP 스트림을 열어 LIMS로 직접 전송한다.
9. 성공하면 `COMPLETED` , 일시 오류면 `RETRY_WAIT` , 영구 오류면 `FAILED` 로 변경한다.
10. 호출 결과를 `report_transfer_attempts` 에 기록한다.

5.3 운영 및 보관 단계

1. 운영자는 전송 상태와 시도 이력을 DB 또는 관리 화면에서 조회한다.
2. `FAILED` 작업은 원인 해결 후 명시적으로 재시도할 수 있다.
3. `PENDING` , `PROCESSING` , `RETRY_WAIT` 파일은 삭제하지 않는다.
4. `COMPLETED` , `FAILED` , `CANCELLED` 파일만 보관 정책에 따라 정리한다.

6. 공유 저장소 계약

6.1 Edge 설정

```
RIST_STORAGE_ROOT=/mnt/rist/reports
RIST_REPORT_STORAGE_KEY=RIST_REPORTS
RIST_REPORT_TRANSFER_MAX_ATTEMPTS=5
```

6.2 Spring Boot 설정 예시

```
rist:
  report-storage:
    roots:
      RIST_REPORTS: /mnt/rist/reports
  report-transfer:
```

```

scheduler-enabled: true
fixed-delay-ms: 5000
batch-size: 10
lease-seconds: 300
heartbeat-seconds: 60
initial-retry-seconds: 30
max-retry-seconds: 1800

```

Windows Spring 서버 예시:

```

rist:
  report-storage:
    roots:
      RIST_REPORTS: 'D:\rist-share'

```

6.3 경로 예시

DB storage_key	RIST_REPORTS
DB package_relative_path	jobs/abc/report/report-package.zip
Edge 실제 경로	/mnt/rist/reports/jobs/abc/report/report-package.zip
Spring 실제 경로	D:\rist-share\jobs\abc\report\report-package.zip

`package_relative_path`는 `/` 구분자를 사용하는 POSIX 상대 경로다. Spring은 OS에 맞는 `Path`로 변환해야 한다.

6.4 경로 보안 규칙

Spring Boot는 다음 항목을 모두 확인해야 한다.

- `storage_key`가 설정에 등록된 허용 키인가
- 상대 경로가 비어 있지 않은가
- `/`, `\`, drive prefix를 이용한 절대 경로가 아닌가
- NUL 문자가 없는가
- 정규화된 경로에 `..` 경로 탈출이 없는가
- `root.resolve(relative).normalize()`가 root 하위인가
- 실제 파일 경로를 계산한 후에도 root의 real path 하위인가
- 일반 파일이며 허용되지 않은 심볼릭 링크가 아닌가

검증 전 경로를 로그에 기록할 때는 제어 문자를 제거한다.

7. DB 구성

운영 흐름은 기존 `jobs`와 신규 3개 테이블로 구성한다.

테이블	소유/기록 주체	목적
jobs	Edge	raw 업로드와 보고서 생성 작업 상태
report_runs	Edge	최종 보고서 패키지 위치와 무결성 메타데이터
report_transfers	Edge 등록, Spring 갱신	LIMS 전송 큐와 최종 상태
report_transfer_attempts	Spring	실제 LIMS 전송 시도 이력

신규 3개 테이블은 현재 운영 DB에 자동으로 존재한다고 가정하면 안 된다. 최초 배포 전에 `edge_api_server/deploy/mariadb_report_queue.sql` 을 DBA가 적용하고 테이블과 외래키 생성 여부를 확인해야 한다.

8. 컬럼 상세

8.1 jobs

기존 테이블을 그대로 사용한다. Spring Boot는 조회만 하고 수정하지 않는다.

컬럼	설명
job_id	Edge 작업 ID
request_number	의뢰번호
experiment_code	실험코드
equipment_code	장비코드
operator_id	실험자/작업자 식별자
status	Edge 작업 상태
completed_at	Edge 작업 완료 시각

8.2 report_runs

컬럼	필수	설명
report_id	Y	UUID 형식 보고서 ID
source_job_id	N	원본 Edge 작업 ID. 웹 보고서는 NULL 가능
request_number	Y	의뢰번호
experiment_code	Y	FT-IR, RAMAN, XRD, TEM 등
equipment_code	Y	실험장비 코드
operator_id	Y	실험자/작업자 식별자
version_no	Y	같은 작업의 보고서 버전. 기본 1

컬럼	필수	설명
generation_status	Y	현재는 READY 사용
storage_key	Y	공유 저장소 논리 키
package_relative_path	Y	저장소 루트 기준 ZIP 상대 경로
package_file_name	Y	일반적으로 report-package.zip
package_size_bytes	Y	ZIP 바이트 크기
package_sha256	Y	소문자 SHA-256 64자리
report_options_json	N	보고서 옵션 JSON 스냅샷
generated_at	Y	Edge 생성 시각, ISO-8601 문자열
created_at	Y	DB 등록 시각
updated_at	Y	DB 변경 시각

제약조건:

- report_id PK
- (storage_key, package_relative_path) UNIQUE
- (source_job_id, version_no) UNIQUE
- source_job_id -> jobs.job_id, 삭제 시 NULL

8.3 report_transfers

컬럼	필수	설명
transfer_id	Y	UUID 형식 전송 ID
report_id	Y	report_runs.report_id
request_number	Y	전송 시점의 의뢰번호 스냅샷
experiment_code	Y	실험코드 스냅샷
equipment_code	Y	장비코드 스냅샷
operator_id	Y	실험자 스냅샷
destination	Y	기본값 LIMS
status	Y	전송 상태
attempt_count	Y	선점된 시도 횟수
max_attempts	Y	최대 시도 횟수
idempotency_key	Y	LIMS 중복 방지와 큐 중복 방지 키
lease_owner	N	현재 처리 중인 Spring worker ID

컬럼	필수	설명
lease_until	N	선점 만료 시각
next_retry_at	N	재시도 가능 시각
requested_at	Y	큐 등록 시각
started_at	N	최초 처리 시작 시각
completed_at	N	최종 성공 시각
external_tracking_id	N	LIMS가 반환한 추적 ID
last_error_code	N	마지막 오류 코드
last_error_message	N	마지막 오류 요약

제약조건:

- transfer_id PK
- (report_id, destination) UNIQUE
- idempotency_key UNIQUE
- report_id -> report_runs.report_id

8.4 report_transfer_attempts

컬럼	필수	설명
attempt_id	Y	AUTO_INCREMENT PK
transfer_id	Y	전송 큐 ID
attempt_no	Y	1부터 시작하는 시도 번호
worker_id	N	처리한 Spring worker ID
started_at	Y	시도 시작 시각
finished_at	N	시도 종료 시각
success	N	성공 TRUE, 실패 FALSE, 진행 중 NULL
response_code	N	LIMS 응답 코드 또는 HTTP 상태
response_message	N	제한된 길이의 응답 요약
error_code	N	내부 오류 코드
error_message	N	민감정보를 제거한 오류 요약
transport_details_json	N	전송 시간 등 소형 메타데이터 JSON

ZIP, 인증 토큰, 전체 응답 본문과 stack trace는 이 테이블에 저장하지 않는다.

9. 상태 모델

9.1 report_runs.generation_status

상태	의미
READY	공유 ZIP 확정과 무결성 메타데이터 계산 완료

현재 Edge는 전송 가능한 보고서만 `report_runs`에 등록한다.

9.2 report_transfers.status

상태	의미	다음 상태
PENDING	최초 전송 대기	PROCESSING, CANCELLED
PROCESSING	worker가 lease를 획득해 처리 중	COMPLETED, RETRY_WAIT, FAILED
RETRY_WAIT	일시 오류 후 대기	PROCESSING, CANCELLED
COMPLETED	LIMS 전달 완료	없음
FAILED	영구 오류 또는 최대 시도 초과	운영자 재처리 시 PENDING
CANCELLED	운영자가 취소	운영자 재처리 시 PENDING

허용되지 않은 상태 전이는 거부하고 운영 로그에 남긴다.

10. Edge 큐 등록 계약

10.1 등록 전 검증

Edge는 DB 등록 전에 다음을 확인한다.

1. 파일이 `RIST_STORAGE_ROOT` 하위인가
2. 일반 파일인가
3. 유효한 ZIP인가
4. 크기를 계산할 수 있는가
5. SHA-256을 계산할 수 있는가
6. 상대 경로가 POSIX 형식이며 `..`를 포함하지 않는가

10.2 등록 트랜잭션

하나의 DB 트랜잭션에서 다음을 수행한다.

1. `report_runs`를 upsert한다.
2. `report_transfers`를 `PENDING`으로 insert한다.
3. 중복 등록 시 기존 전송 행을 재사용하고 완료 상태를 되돌리지 않는다.

4. 두 작업 중 하나라도 실패하면 전체를 rollback한다.

큐 등록 후에만 `jobs.status=COMPLETED` 로 변경한다. 큐 등록 실패 시 Edge 작업은 `FAILED`, 오류 코드는 `REPORT_QUEUE_REGISTRATION_FAILED` 가 된다.

11. Spring Boot 구현 계약

11.1 권장 컴포넌트

ReportTransferProperties

- 저장소 키/루트 매핑, 스케줄, lease, retry 설정

ReportTransferRepository

- 선점, 상태 전이, attempt 기록 SQL

SharedReportPackageResolver

- 상대 경로 해석, 경로 탈출 차단, 크기/해시/ZIP 검증

LimsReportSender

- 검증된 ZIP 스트림과 업무 메타데이터를 LIMS에 전송

ReportTransferWorker

- 선점 -> 검증 -> 전송 -> 결과 저장 오케스트레이션

ReportTransferScheduler

- fixed delay로 worker 실행

11.2 설정 클래스 예시

```
@ConfigurationProperties(prefix = "rist")
public record RistTransferProperties(
    ReportStorage reportStorage,
    ReportTransfer reportTransfer
) {
    public record ReportStorage(Map<String, Path> roots) {}
    public record ReportTransfer(
        boolean schedulerEnabled,
        long fixedDelayMs,
        int batchSize,
        long leaseSeconds,
        long heartbeatSeconds,
        long initialRetrySeconds,
        long maxRetrySeconds
    ) {}
}
```

설정 시작 시 저장소 키가 중복되지 않는지, 루트가 절대 경로인지, 읽기 가능한지 검증한다.

12. 큐 선점

12.1 선점 대상

- `PENDING`
- `next_retry_at` 이 지난 `RETRY_WAIT`
- Spring 비정상 종료로 lease가 만료된 `PROCESSING`
- `attempt_count < max_attempts`

12.2 MariaDB 선점 SQL

한 건 선점 예시:

```
START TRANSACTION;

SELECT transfer_id
FROM report_transfers
WHERE attempt_count < max_attempts
  AND (
    status = 'PENDING'
    OR (status = 'RETRY_WAIT'
        AND next_retry_at <= CURRENT_TIMESTAMP(6))
    OR (status = 'PROCESSING'
        AND lease_until < CURRENT_TIMESTAMP(6))
  )
ORDER BY requested_at, transfer_id
LIMIT 1
FOR UPDATE SKIP LOCKED;

UPDATE report_transfers
SET status = 'PROCESSING',
    attempt_count = attempt_count + 1,
    lease_owner = :workerId,
    lease_until = DATE_ADD(
      CURRENT_TIMESTAMP(6), INTERVAL :leaseSeconds SECOND
    ),
    started_at = COALESCE(started_at, CURRENT_TIMESTAMP(6)),
    next_retry_at = NULL,
    last_error_code = NULL,
    last_error_message = NULL
WHERE transfer_id = :transferId;

INSERT INTO report_transfer_attempts (
  transfer_id,
  attempt_no,
  worker_id
) SELECT
  transfer_id,
  attempt_count,
```

```

:workerId
FROM report_transfers
WHERE transfer_id = :transferId;

COMMIT;

```

파일 검증과 LIMS 전송은 **COMMIT** 이후에 실행한다. **batch-size** 만큼 반복해서 선점할 수 있지만, 한 트랜잭션에서 여러 건을 오랫동안 잠그지 않는다.

12.3 worker ID

lease_owner 는 인스턴스와 실행 스레드를 구분할 수 있어야 한다.

```
{applicationName}:{hostName}:{instanceId}:{threadName}
```

최대 128자로 제한한다.

13. Lease와 비정상 종료 복구

- 기본 lease는 300초를 권장한다.
- 예상 전송 시간이 lease의 절반을 넘으면 heartbeat로 연장한다.
- heartbeat 갱신에는 반드시 **transfer_id**, **status=PROCESSING**, **lease_owner** 조건을 사용한다.
- 현재 worker가 lease를 잃으면 성공/실패 상태를 갱신하지 말고 처리를 중단한다.
- lease가 만료된 **PROCESSING** 은 다른 worker가 다시 선점할 수 있다.

```

UPDATE report_transfers
SET lease_until = DATE_ADD(
    CURRENT_TIMESTAMP(6), INTERVAL :leaseSeconds SECOND
)
WHERE transfer_id = :transferId
  AND status = 'PROCESSING'
  AND lease_owner = :workerId;

```

갱신 행 수가 0이면 lease를 상실한 것으로 처리한다.

14. 공유 ZIP 검증

14.1 조회 SQL

```

SELECT
    t.transfer_id,
    t.report_id,
    t.request_number,
    t.experiment_code,

```

```

t.equipment_code,
t.operator_id,
t.destination,
t.attempt_count,
t.max_attempts,
t.idempotency_key,
r.storage_key,
r.package_relative_path,
r.package_file_name,
r.package_size_bytes,
r.package_sha256,
r.generated_at
FROM report_transfers t
JOIN report_runs r ON r.report_id = t.report_id
WHERE t.transfer_id = :transferId
AND t.status = 'PROCESSING'
AND t.lease_owner = :workerId;

```

14.2 파일 검증 순서

1. `storage_key` 를 설정된 루트에 매핑한다.
2. 상대 경로를 OS `Path` 로 변환한다.
3. 정규화 전후에 절대 경로와 경로 탈출을 차단한다.
4. root와 파일의 real path 관계를 검증한다.
5. 일반 파일과 읽기 권한을 확인한다.
6. 실제 크기와 `package_size_bytes` 를 비교한다.
7. 스트리밍 SHA-256과 `package_sha256` 을 비교한다.
8. ZIP central directory를 열어 유효성을 확인한다.
9. 필요하면 ZIP entry의 절대 경로와 `..` 를 검사한다.

SHA-256은 파일 전체를 메모리에 올리지 않고 버퍼 스트리밍으로 계산한다.

14.3 ZIP 사용 방식

검증된 ZIP을 현재 공유 경로에서 입력 스트림으로 열어 LIMS에 전송한다. Spring Boot 전용 영구 수신 폴더로 복사하지 않는다. 사용하는 HTTP client가 임시 파일을 요구하는 경우 작업 종료 시 즉시 제거되는 OS 임시 파일만 허용한다.

15. LIMS 전송 어댑터 계약

Spring 내부 인터페이스 권장안:

```

public interface LimsReportSender {
    LimsTransferResult send(
        ReportTransferContext context,
        Path packagePath
    );
}

```

```
public record LimsTransferResult(
    boolean success,
    boolean retryable,
    String responseCode,
    String responseMessage,
    String externalTrackingId
) {}
```

LIMS 요청에는 최소한 다음 업무키가 포함되어야 한다.

- 의뢰번호 `request_number`
- 실험코드 `experiment_code`
- 장비코드 `equipment_code`
- 실험자 `operator_id`
- 최종 보고서 ZIP
- 가능하면 `idempotency_key`

LIMS가 idempotency를 지원하면 Edge에서 생성한 `idempotency_key`를 그대로 전달한다. timeout 후 LIMS 접수 여부가 불명확한 경우 중복 전송을 막기 위해 LIMS 추적 ID 조회 또는 idempotency 조회 절차가 필요하다.

16. 성공 처리

LIMS 성공 응답 후 같은 DB 트랜잭션에서 attempt와 큐를 완료한다.

```
START TRANSACTION;

UPDATE report_transfer_attempts
SET finished_at = CURRENT_TIMESTAMP(6),
    success = TRUE,
    response_code = :responseCode,
    response_message = :responseMessage
WHERE transfer_id = :transferId
    AND attempt_no = :attemptNo;

UPDATE report_transfers
SET status = 'COMPLETED',
    completed_at = CURRENT_TIMESTAMP(6),
    external_tracking_id = :trackingId,
    lease_owner = NULL,
    lease_until = NULL,
    next_retry_at = NULL,
    last_error_code = NULL,
    last_error_message = NULL
WHERE transfer_id = :transferId
    AND status = 'PROCESSING'
    AND lease_owner = :workerId;
```

```
COMMIT;
```

두 번째 UPDATE가 0건이면 lease 상실 가능성이 있으므로 성공 상태를 임의로 덮어쓰지 말고 운영 오류를 기록한다.

17. 오류 분류와 재시도

17.1 재시도 가능 오류

조건	권장 오류 코드
연결 실패	LIMS_CONNECTION_FAILED
connect/read timeout	LIMS_TIMEOUT
LIMS HTTP 429	LIMS_RATE_LIMITED
LIMS 일시적 5xx	LIMS_SERVER_ERROR
공유 저장소 일시적 I/O 오류	REPORT_STORAGE_TEMPORARY_ERROR

17.2 즉시 실패 오류

조건	권장 오류 코드
알 수 없는 storage key	REPORT_STORAGE_KEY_UNKNOWN
경로 탈출 또는 절대 경로	REPORT_PATH_INVALID
파일 없음	REPORT_PACKAGE_NOT_FOUND
크기 불일치	REPORT_PACKAGE_SIZE_MISMATCH
SHA-256 불일치	REPORT_PACKAGE_HASH_MISMATCH
ZIP 손상	REPORT_PACKAGE_INVALID_ZIP
파일 읽기 권한 없음	REPORT_PACKAGE_ACCESS_DENIED
LIMS 인증/인가 실패	LIMS_AUTH_FAILED
의뢰번호 등 업무키 오류	LIMS_BUSINESS_KEY_REJECTED
LIMS가 파일 형식을 거부	LIMS_PACKAGE_REJECTED

인증 실패가 토큰 갱신으로 해결되는 구조라면 한 번 갱신한 뒤 재판정할 수 있다.

17.3 Backoff

권장 계산식:

```
delay = min(maxRetrySeconds,
             initialRetrySeconds * 2^(attempt_count - 1))
```

```

jitter = delay의 0~20%
next_retry_at = now + delay + jitter

```

최대 횟수에 도달하면 **FAILED** 로 전환한다.

17.4 실패 처리 SQL

```

START TRANSACTION;

UPDATE report_transfer_attempts
SET finished_at = CURRENT_TIMESTAMP(6),
    success = FALSE,
    response_code = :responseCode,
    response_message = :responseMessage,
    error_code = :errorCode,
    error_message = :errorMessage
WHERE transfer_id = :transferId
    AND attempt_no = :attemptNo;

UPDATE report_transfers
SET status = :nextStatus,
    next_retry_at = :nextRetryAt,
    lease_owner = NULL,
    lease_until = NULL,
    last_error_code = :errorCode,
    last_error_message = :errorMessage
WHERE transfer_id = :transferId
    AND status = 'PROCESSING'
    AND lease_owner = :workerId;

COMMIT;

```

nextStatus 는 **RETRY_WAIT** 또는 **FAILED** 다. **FAILED** 일 때 **next_retry_at=NULL** 로 저장한다.

18. 멍등성과 중복 방지

1. Edge는 **(report_id, destination)** UNIQUE로 중복 큐를 막는다.
2. **idempotency_key** 도 UNIQUE로 저장한다.
3. Spring은 같은 **transfer_id** 에 대해 lease 없이 전송하지 않는다.
4. LIMS가 지원하면 **idempotency_key** 를 전송 요청에 포함한다.
5. 성공 응답 후 DB 갱신이 실패하면 LIMS 접수 상태를 조회한 뒤 재처리한다.
6. **COMPLETED** 작업을 자동으로 **PENDING** 으로 되돌리지 않는다.

19. 운영 SQL

19.1 대기 및 처리 중 작업

```
SELECT transfer_id, report_id, request_number, experiment_code,
       status, attempt_count, max_attempts,
       requested_at, next_retry_at, lease_owner, lease_until
FROM report_transfers
WHERE status IN ('PENDING', 'PROCESSING', 'RETRY_WAIT')
ORDER BY requested_at;
```

19.2 lease 만료 작업

```
SELECT transfer_id, lease_owner, lease_until, attempt_count
FROM report_transfers
WHERE status = 'PROCESSING'
      AND lease_until < CURRENT_TIMESTAMP(6);
```

19.3 의뢰번호별 최종 상태

```
SELECT r.request_number, r.experiment_code,
       r.package_relative_path, t.status,
       t.attempt_count, t.completed_at,
       t.external_tracking_id,
       t.last_error_code, t.last_error_message
FROM report_runs r
JOIN report_transfers t ON t.report_id = r.report_id
WHERE r.request_number = :requestNumber
ORDER BY r.generated_at DESC;
```

19.4 전송 시도 이력

```
SELECT attempt_no, worker_id, started_at, finished_at,
       success, response_code, error_code, error_message
FROM report_transfer_attempts
WHERE transfer_id = :transferId
ORDER BY attempt_no;
```

19.5 운영자 수동 재시도

원인을 해결하고 전송 결과가 **COMPLETED** 가 아님을 확인한 뒤 실행한다.

```
UPDATE report_transfers
SET status = 'PENDING',
    attempt_count = 0,
    next_retry_at = NULL,
```



```

lease_owner = NULL,
lease_until = NULL,
completed_at = NULL,
last_error_code = NULL,
last_error_message = NULL
WHERE transfer_id = :transferId
AND status IN ('FAILED', 'CANCELLED');

```

운영 도구에서 실행한 사용자, 시각과 사유를 별도 감사 로그에 남긴다.

20. 로그와 모니터링

20.1 필수 로그 필드

- transfer_id
- report_id
- request_number
- experiment_code
- attempt_no
- worker_id
- status_before, status_after
- error_code
- 처리 시간

ZIP 경로 전체, 인증 토큰, 개인정보와 LIMS 전체 응답은 로그에 남기지 않는다.

20.2 권장 지표

- 상태별 큐 개수
- 가장 오래된 PENDING 대기 시간
- 성공/실패/재시도 건수
- 평균 및 p95 전송 시간
- lease 만료 복구 건수
- 파일 없음, 크기 불일치, 해시 불일치 건수

20.3 운영 알림 기준

- PENDING 최고 대기 시간이 기준 초과
- 연속 LIMS 인증 실패
- 동일 오류 코드 급증
- lease 만료 작업 반복 발생
- 파일 무결성 오류 발생

21. 보관과 삭제 정책

- `PENDING`, `PROCESSING`, `RETRY_WAIT` ZIP은 삭제하지 않는다.
- `COMPLETED`, `FAILED`, `CANCELLED` 만 보관 기간 정책 대상이다.
- 파일 삭제 전 `report_runs` 와 전송 이력의 감사 보존 기간을 확인한다.
- 파일을 먼저 삭제하면 Spring은 `REPORT_PACKAGE_NOT_FOUND` 로 실패 처리한다.
- 동일 상대 경로의 파일을 다른 내용으로 덮어쓰지 않는다.
- 재생성 보고서는 새 버전과 새 상대 경로를 사용한다.

22. 배포 순서

1. Edge와 Spring 서버에 공유 저장소를 마운트한다.
2. 같은 저장소에 쓰기/읽기 테스트 파일로 접근을 확인한다.
3. MariaDB 백업과 변경 승인 절차를 수행한다.
4. `edge_api_server/deploy/mariadb_report_queue.sql` 을 적용한다.
5. 신규 3개 테이블, 인덱스와 외래키를 확인한다.
6. Spring에 storage key 매핑과 scheduler 설정을 배포한다.
7. Spring scheduler를 비활성화한 상태로 기동해 DB/저장소 연결을 점검한다.
8. Edge 최신 버전을 배포하고 테스트 보고서 한 건을 큐에 등록한다.
9. Spring scheduler를 활성화한다.
10. LIMS 성공과 DB 상태/attempt 이력을 확인한다.
11. 장애 복구와 재시도 테스트 후 운영 전환한다.

23. 배포 확인 SQL

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = DATABASE()
  AND table_name IN (
    'jobs',
    'report_runs',
    'report_transfers',
    'report_transfer_attempts'
  )
ORDER BY table_name;
```

네 행이 모두 조회되어야 한다.

```
SHOW CREATE TABLE report_runs;
SHOW CREATE TABLE report_transfers;
SHOW CREATE TABLE report_transfer_attempts;
```

24. 인수 테스트

번호	시나리오	기대 결과
1	정상 ZIP 등록	<code>report_runs=READY</code> , <code>report_transfers=PENDING</code>
2	Spring 정상 전송	transfer와 attempt가 성공 완료
3	같은 보고서 재등록	중복 큐가 생성되지 않음
4	두 Spring 인스턴스 동시 실행	한 transfer를 한 worker만 선점
5	worker 강제 종료	lease 만료 후 다른 worker가 복구
6	LIMS timeout	<code>RETRY_WAIT</code> , backoff 후 재시도
7	최대 시도 초과	<code>FAILED</code> 전환
8	파일 삭제	<code>REPORT_PACKAGE_NOT_FOUND</code>
9	파일 내용 변경	크기 또는 SHA-256 불일치로 실패
10	손상 ZIP	<code>REPORT_PACKAGE_INVALID_ZIP</code>
11	<code>../</code> 경로 주입	파일 접근 전 <code>REPORT_PATH_INVALID</code>
12	알 수 없는 storage key	<code>REPORT_STORAGE_KEY_UNKNOWN</code>
13	LIMS 401/403	<code>LIMS_AUTH_FAILED</code> 최종 실패
14	LIMS 429/5xx	재시도 후 성공 또는 최대 시도 실패
15	운영자 수동 재시도	감사 로그와 새 attempt 생성

25. 제거된 HTTP 계약

다음 항목은 더 이상 사용하지 않는다.

- `POST /api/v1/edge/reports`
- `multipart/form-data` 방식의 Edge -> Spring ZIP 전달
- Spring Boot의 ZIP 수신 및 중간 저장 API
- `RIST_SPRING_CALLBACK_URL`
- `RIST_SPRING_CALLBACK_TIMEOUT_SECONDS`
- `RIST_SPRING_CALLBACK_MAX_ATTEMPTS`
- `CALLBACK_PENDING` 작업 상태

실험 PC/C# 프로그램의 기존 Edge 업로드 API에는 변화가 없다. C# 프로그램은 Spring Boot와 전송 큐를 직접 호출하지 않고 `GET /api/v1/jobs/{jobId}` 로 Edge 보고서 생성 완료까지만 확인한다.

26. Spring Boot 개발 완료 기준

다음 조건을 모두 만족해야 연동 완료로 본다.

- 신규 DDL 적용과 시작 시 스키마 검증
- 다중 인스턴스 안전한 큐 선택
- lease 연장과 비정상 종료 복구
- storage key 기반 경로 해석
- 경로 탈출, 크기, SHA-256, ZIP 검증
- 공유 ZIP 직접 스트리밍 전송
- 성공, 재시도, 최종 실패 상태 전이
- 모든 호출의 attempt 이력 저장
- 먹등키 또는 LIMS 중복 방지 절차
- 운영 조회, 알림과 수동 재처리 기능
- 인수 테스트 15개 통과

부록 A. 전체 DDL

동일한 SQL은 `edge_api_server/deploy/mariadb_report_queue.sql`에 있다. 기존 `jobs`가 같은 MariaDB schema에 존재하는 것을 전제로 한다.

```
CREATE TABLE IF NOT EXISTS report_runs (
  report_id VARCHAR(36) NOT NULL,
  source_job_id VARCHAR(36),
  request_number VARCHAR(128) NOT NULL,
  experiment_code VARCHAR(64) NOT NULL,
  equipment_code VARCHAR(64) NOT NULL,
  operator_id VARCHAR(100) NOT NULL,
  version_no INT NOT NULL DEFAULT 1,
  generation_status VARCHAR(32) NOT NULL,
  storage_key VARCHAR(64) NOT NULL DEFAULT 'RIST_REPORTS',
  package_relative_path VARCHAR(512) NOT NULL,
  package_file_name VARCHAR(255) NOT NULL,
  package_size_bytes BIGINT NOT NULL,
  package_sha256 CHAR(64) NOT NULL,
  report_options_json LONGTEXT,
  generated_at VARCHAR(64) NOT NULL,
  created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  updated_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6)
    ON UPDATE CURRENT_TIMESTAMP(6),
  PRIMARY KEY (report_id),
  UNIQUE KEY uq_report_runs_package_path (
    storage_key,
    package_relative_path
  ),
  UNIQUE KEY uq_report_runs_job_version (source_job_id, version_no),
```

```

    KEY idx_report_runs_request (request_number, experiment_code),
    CONSTRAINT fk_report_runs_job FOREIGN KEY (source_job_id)
        REFERENCES jobs(job_id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS report_transfers (
    transfer_id VARCHAR(36) NOT NULL,
    report_id VARCHAR(36) NOT NULL,
    request_number VARCHAR(128) NOT NULL,
    experiment_code VARCHAR(64) NOT NULL,
    equipment_code VARCHAR(64) NOT NULL,
    operator_id VARCHAR(100) NOT NULL,
    destination VARCHAR(64) NOT NULL DEFAULT 'LIMS',
    status VARCHAR(32) NOT NULL,
    attempt_count INT NOT NULL DEFAULT 0,
    max_attempts INT NOT NULL DEFAULT 5,
    idempotency_key VARCHAR(128) NOT NULL,
    lease_owner VARCHAR(128),
    lease_until DATETIME(6),
    next_retry_at DATETIME(6),
    requested_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    started_at DATETIME(6),
    completed_at DATETIME(6),
    external_tracking_id VARCHAR(255),
    last_error_code VARCHAR(128),
    last_error_message TEXT,
    created_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
    updated_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6)
        ON UPDATE CURRENT_TIMESTAMP(6),
    PRIMARY KEY (transfer_id),
    UNIQUE KEY uq_report_transfers_report_destination (
        report_id,
        destination
    ),
    UNIQUE KEY uq_report_transfers_idempotency (idempotency_key),
    KEY idx_report_transfers_scheduler (
        status,
        next_retry_at,
        lease_until,
        requested_at
    ),
    CONSTRAINT fk_report_transfers_report FOREIGN KEY (report_id)
        REFERENCES report_runs(report_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE IF NOT EXISTS report_transfer_attempts (
    attempt_id BIGINT NOT NULL AUTO_INCREMENT,
    transfer_id VARCHAR(36) NOT NULL,
    attempt_no INT NOT NULL,
    worker_id VARCHAR(128),
    started_at DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),

```

```

finished_at DATETIME(6),
success BOOLEAN,
response_code VARCHAR(128),
response_message TEXT,
error_code VARCHAR(128),
error_message TEXT,
transport_details_json LONGTEXT,
PRIMARY KEY (attempt_id),
UNIQUE KEY uq_report_transfer_attempt (transfer_id, attempt_no),
KEY idx_report_transfer_attempts_transfer (
    transfer_id,
    started_at
),
CONSTRAINT fk_report_transfer_attempts_transfer
    FOREIGN KEY (transfer_id)
    REFERENCES report_transfers(transfer_id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

운영 적용 전 개발 DB에서 먼저 실행하고 외래키, 인덱스와 문자셋을 확인한다.